

Title: A Proposal to Simplify the Unified Executors Design
Authors: Chris Kohlhoff, chris@kohlhoff.com
Jared Hoberock, jhoberock@nvidia.com
Chris Mysen, mysen@google.com
Gordon Brown, gordon@codeplay.com
Document Number: P0688R0
Date: 2017-06-19
Audience: SG1 - Concurrency and Parallelism
Reply-to: sg1-exec@googlegroups.com

At the Kona meeting of the ISO C++ Standards Committee in 2017, we presented a design for executors, which we envision to be components for creating execution in C++. Our design, described in [P0443R1](#), was a unification of three independent proposals targeted at different use cases. Those use cases are the execution-creating interfaces of the Standard Library (e.g., `async`), as well as interfaces found in the Concurrency, Parallelism, and Networking Technical Specifications. We believe P0443R1's functionality is necessary to support them.

Discussion in Kona made it clear that the design of P0443R1 was too complex. We agree, and have considered possible simplifications. One approach toward reducing complexity would discard functionality. However, we do not believe it is possible to eliminate functionality without compromising interoperability with the use cases we have committed to supporting. Moreover, simply eliminating features would only reduce complexity today. Those features and the complexity they bring could return tomorrow.

Instead, we believe a successful approach will be to refactor P0443R1's functionality into a more manageable, factored form based on *executor properties*, which are user-requestable behaviors which modify the way executors create execution. Compared to P0443R1's design, we believe a property-based design will be simpler both for executor clients to use and for executor authors to implement. Because our properties design is open-ended, it may be extended with new properties. Finally, this design's specification is much more compact than P0443R1. As a consequence, we have been able to quickly produce an [open source prototype](#) and [several example programs](#).

Introductory examples. To briefly summarize by example, higher-level, indirect use of executors through control structures remains the same as with P0443R1:

```
// execute an async on an executor:  
auto future = std::async(my_executor, task1);  
  
// execute a parallel for_each on an executor:  
std::for_each(std::execution::par.on(my_executor), data.begin(), data.end(), task2);
```

Lower-level, direct use of executors through execution functions changes to use requirements and preferences:

```
// make require(), prefer(), and properties available  
using namespace std::experimental::execution;  
  
// execute a non-blocking, fire-and-forget task on an executor:  
require(my_executor, oneway, never_blocking).execute(task1);  
  
// execute a non-blocking, two-way task on an executor. prefer to execute as a continuation:  
auto future2 = prefer(require(my_executor, twoway), continuation).twoway_execute(task2);  
  
// when future is ready, execute a possibly-blocking billion-agent task  
auto bulk_exec = require(my_executor, possibly_blocking, bulk, then);  
auto future3 = bulk_exec.bulk_then_execute(task3, 1<<30, future2, result_factory, shared_factory);
```

1 Proposed Simplification

The complexity of P0443R1's design emanates from the execution functions comprising the fundamental API for creating work with executors. This set of functions is the result of the cross product of three sets of properties: blocking behavior (i.e., never-blocking, possibly-blocking, and always-blocking), cardinality (i.e. single and bulk), and directionality (one-way and two-way). A partial cross product yields the set of sixteen execution functions identified by P0443R1. Rather than "hard-code" a set of arbitrary combinations into the possible executor behaviors, our proposed simplification allows the user to programmatically build an executor with the desired behavior. This refactoring allows us to reduce the set of execution functions.

1.1 Execution Functions

Our simplified design defines six fundamental execution functions: `execute`, `twoway_execute`, `then_execute`, `bulk_execute`, `bulk_twoway_execute`, and `bulk_then_execute`. Each of these functions has a unique signature distinguishing them. Unlike P0443R1's design, execution functions are always executor member functions and they do not have corresponding customization points. Otherwise, these execution functions retain their original meanings from P0443R1.

P0443R1 defined a suite of customization points which adapt an executor's native behavior to provide the execution semantics of the customization point. The intention was to provide a uniform user experience regardless of the native behavior of the executor. Because execution functions are no longer customization points, this adaptation must be performed through other means. The mechanism we propose to perform this adaptation is instead provided by two new customization points: `execution::require()` and `execution::prefer()`. These customization points provide a path to state user requirements and preferences separately from calling a fundamental execution function.

1.2 User Requirements

For example, suppose a user requires to create an execution agent with two-way, non-blocking execution. Under the API specified by P0443R1, this is accomplished by using the specific customization point tasked with implementing this combination of properties:

```
using namespace std::experimental::execution;
auto future = async_post(exec, task);
```

In our proposed design, the user separately *requires* the properties of interest, and then calls an execution function:

```
using namespace std::experimental::execution;
auto future = require(exec, twoway, never_blocking).twoway_execute(task);
```

The `require()` call returns a new executor adapting the given executor's native behavior to guarantee the required behaviors. If the given executor's native behavior already provides the required guarantee, then `require()` behaves like the identity function and returns the executor unchanged. If it is not possible to satisfy a requirement, then it is a compile-time error.

As another example, suppose a user requires to create an execution agent with one-way, blocking execution. This is not possible with P0443R1's API, because it does not specify an execution function for this combination of requirements. However, our new proposed design does permit this combination:

```
using namespace std::experimental::execution;
require(exec, oneway, always_blocking).execute(task);
```

1.3 User Preferences

Some of the properties users desire of executors are not hard requirements. Instead, they are softer *preferences*. For example, suppose a user requires to create a one-way, never-blocking execution agent and for performance reasons, would prefer that the agent execute as a continuation of the calling thread. P0443R1 defines a special execution function named `defer()` for this purpose:

```
using namespace std::experimental::execution;
defer(exec, task);
```

As described by P0443R1, `defer()`'s semantics are equivalent to `post()`'s. The distinction is that `defer()` acts as a hint to the executor to create the execution agent in a particular way. The presence of `defer()` in P0443R1 was controversial.

Our new proposed design introduces `prefer()` as an avenue for communicating such hints:

```
using namespace std::experimental::execution;
require(exec, oneway).prefer(is_continuation).execute(task);
```

Unlike requirements, executors are under no obligation to satisfy user preferences. If it is not possible to satisfy a preference, then it is not a compile-time error.

`execution::require()` and `execution::prefer()` are both customization points, and they are the only two customization points in our design.

1.4 Executor Properties

In our proposed design, executor properties are objects associated with an executor. Through calls to `require()` and `prefer()`, users may either strongly or weakly reassociate a property with a given executor. Such reassociations may transform the executor's type in the process. We believe a design for executor properties should have the following characteristics.

Extensible. The design should permit the creation of new properties, by both user code and future additions to the standard.

Forwardable. A control structure may pass an executor directly to another control structure. For example:

```
template<class Executor, class Function>
void my_control_structure(const Executor& ex, Function f)
{
    // add the oneway property
    oneway_ex = execution::require(ex, execution::oneway).execute(f);
    my_other_control_structure(oneway_ex, f);
}
```

The properties associated with `ex` should still be available to `my_other_control_structure`.

Overridable. A control structure may introduce properties to an executor before passing it to another. The first control structure may be aware of certain properties and wish to override them:

```
template<class Executor, class Function>
void my_control_structure(const Executor& ex, Function f)
{
    // override the blocking property
    never_blocking_ex = execution::require(ex, execution::never_blocking);
    my_other_control_structure(never_blocking_ex, f);
}
```

1.4.1 Proposed Properties

Our proposal includes eight sets of properties we have identified as necessary to supporting the needs of the Standard Library and other technical specifications. Two of these sets describe the directionality and cardinality of execution member functions which create execution. When a user requests these properties, they are implicitly requesting an executor which provides the execution functions implied by the request.

Directionality. The directionality properties we propose are oneway, twoway, and then. An executor with the oneway property has either or both of the one-way execution functions: `.execute()` or `.bulk_execute()`. An executor with the twoway property has either or both of the two-way execution functions: `.twoway_execute()` or `.bulk_twoway_execute()`. An executor with the then property has either or both of the then_ execution functions: `then_execute()` or `bulk_then_execute()`. Because a single executor type can have one or more of these member functions all at once, these properties are not mutually exclusive.

Cardinality. There are two cardinality properties: `single` and `bulk`. An executor with the `single` property has at least one execution function which creates a single execution agent from a single call. Likewise, an executor with the `bulk` property has at least one execution function which creates multiple execution agents in bulk from a single call. Like the directionality properties, the cardinality properties are not mutually exclusive, because it is possible for a single executor type to have both kinds of execution functions.

Unlike the directionality and cardinality properties, which imply the existence of certain execution functions, other properties modify the behavior of those functions.

Blocking. There are three mutually-exclusive blocking properties : `never_blocking`, `possibly_blocking`, and `always_blocking`. The blocking properties guarantee the blocking behavior of an executor's execution functions. For example, when `.execute(task)` is called on an executor whose blocking property is `never_blocking`, then the forward progress of the calling thread will never be blocked pending the completion of the execution agent created by the call. The same guarantee holds for every other execution function of that executor. The net effect is that, unlike in P0443R1, the blocking behavior of execution functions is completely a property of the executor type. However, that property can be changed at will by transforming the executor into a different type through a call to `require()`.

Continuations. There are two mutually-exclusive properties for indicate that a task submitted to an executor represents a continuation of the calling thread: `continuation` and `not_continuation`. A client may use the `continuation` property to indicate that a program may execute more efficiently when tasks are executed as continuations.

Future task submission. There are two mutually-exclusive properties to indicate the likelihood of additional task submission in the future. The `outstanding_work` work property indicates to an executor that additional task submission is likely. Likewise, the `not_outstanding_work` property indicates that no outstanding work remains.

Bulk forward progress guarantees. There are three mutually exclusive properties which describe the forward progress guarantees of execution agents created in bulk. These describe the forward progress of an agent with respect to the other agents created in the same submission. These are `bulk_sequenced_execution`, `bulk_parallel_execution`, and `bulk_unsequenced_execution`, and they correspond to the three standard execution policies.

Thread execution mapping guarantees. There are two mutually exclusive properties for describing the way in which execution agents are mapped onto threads. `thread_execution_mapping` guarantees that execution agents are mapped onto threads of execution, while `new_thread_execution_mapping` extends that guarantee by guaranteeing that each execution agent will be executed on a newly-created individual thread. These guarantees may be used by the client to reason about the existence and sharing of thread-local storage over an execution agent's lifetime.

Allocators. A final property, `allocator`, associates an allocator with an executor. A client may use this property to require the use of a preferred allocator when allocating storage necessary to create execution. Of the properties we propose to introduce, `allocator(alloc)` is the only one which takes an additional parameter; namely, the desired allocator to use.

2 Acknowledgements

This proposal's design benefited greatly from feedback provided by Hans Boehm, Carter Edwards, Michael Garland, Thomas Heller, David Hollman, Lee Howes, Bryce Lebach, Hartmut Kaiser, Gor Nishanov, Billy O'Neal, Torvald Riegel, Thomas Rodgers, and Michael Wong.

3 Approximate Proposed Wording

3.0.1 Header <execution> synopsis

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {
namespace execution {

    // Directionality properties:

    constexpr struct oneway_t {} oneway;
    constexpr struct twoway_t {} twoway;
    constexpr struct then_t {} then;

    // Cardinality properties:

    constexpr struct single_t {} single;
```

```

constexpr struct bulk_t {} bulk;

// Blocking properties:

constexpr struct never_blocking_t {} never_blocking;
constexpr struct possibly_blocking_t {} possibly_blocking;
constexpr struct always_blocking_t {} always_blocking;

// Properties to indicate if submitted tasks represent continuations:

constexpr struct continuation_t {} continuation;
constexpr struct not_continuation_t {} not_continuation;

// Properties to indicate likely task submission in the future:

constexpr struct outstanding_work_t {} outstanding_work;
constexpr struct not_outstanding_work_t {} not_outstanding_work;

// Properties for bulk execution forward progress guarantees:

constexpr struct bulk_sequenced_execution_t {} bulk_sequenced_execution;
constexpr struct bulk_parallel_execution_t {} bulk_parallel_execution;
constexpr struct bulk_unsequenced_execution_t {} bulk_unsequenced_execution;

// Properties for mapping of execution on to threads:

constexpr struct thread_execution_mapping_t {} thread_execution_mapping;
constexpr struct new_thread_execution_mapping_t {} new_thread_execution_mapping;

// Memory allocation properties:

template<class ProtoAllocator> struct allocator_t { ProtoAllocator alloc; };
template<class ProtoAllocator> constexpr allocator_t<ProtoAllocator> allocator(const ProtoAllocator& a) { return {a}; }

// Executor type traits:

template<class Executor> struct is_executor;
template<class Executor> struct is_oneway_executor;
template<class Executor> struct is_twoway_executor;
template<class Executor> struct is_then_executor;
template<class Executor> struct is_bulk_oneway_executor;
template<class Executor> struct is_bulk_twoway_executor;
template<class Executor> struct is_bulk_then_executor;

template<class Executor> constexpr bool is_executor_v = is_executor<Executor>::value;
template<class Executor> constexpr bool is_oneway_executor_v = is_oneway_executor<Executor>::value;
template<class Executor> constexpr bool is_twoway_executor_v = is_twoway_executor<Executor>::value;
template<class Executor> constexpr bool is_then_executor_v = is_then_executor<Executor>::value;
template<class Executor> constexpr bool is_bulk_oneway_executor_v = is_bulk_oneway_executor<Executor>::value;
template<class Executor> constexpr bool is_bulk_twoway_executor_v = is_bulk_twoway_executor<Executor>::value;
template<class Executor> constexpr bool is_bulk_then_executor_v = is_bulk_then_executor<Executor>::value;

template<class Executor> struct executor_context;
template<class Executor, class T> struct executor_future;
template<class Executor> struct executor_shape;
template<class Executor> struct executor_index;

template<class Executor> using executor_context_t = typename executor_context<Executor>::type;
template<class Executor, class T> using executor_future_t = typename executor_future<Executor, T>::type;
template<class Executor> using executor_shape_t = typename executor_shape<Executor>::type;
template<class Executor> using executor_index_t = typename executor_index<Executor>::type;

// Member detection type traits for properties:

template<class Executor, class Property> struct has_require_member;
template<class Executor, class Property> struct has_prefer_member;

template<class Executor, class Property>
    constexpr bool has_require_member_v = has_require_member<Executor, Property>::value;
template<class Executor, class Property>
    constexpr bool has_prefer_member_v = has_prefer_member<Executor, Property>::value;

// Member return type traits for properties:

template<class Executor, class Property> struct require_member_result;
template<class Executor, class Property> struct prefer_member_result;

template<class Executor, class Property>
    using require_member_result_t = typename require_member_result<Executor, Property>::type;
template<class Executor, class Property>
    using prefer_member_result_t = typename prefer_member_result<Executor, Property>::type;

// Customization points:

namespace {
    constexpr unspecified require = unspecified;

```

```

    constexpr unspecified prefer = unspecified;
}

// Customization point type traits:

template<class Executor, class... Properties> struct can_require;
template<class Executor, class... Properties> struct can_prefer;

template<class Executor, class... Properties>
    constexpr bool can_require_v = can_require<Executor, Properties>::value;
template<class Executor, class... Properties>
    constexpr bool can_prefer_v = can_prefer<Executor, Properties>::value;

// Polymorphic executor wrappers:

class bad_executor;
class executor;

} // namespace execution
} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

3.1 Requirements

3.1.1 Customization point objects

(The following text has been adapted from the draft Ranges Technical Specification.)

A *customization point object* is a function object (C++ Std, [function.objects]) with a literal class type that interacts with user-defined types while enforcing semantic requirements on that interaction.

The type of a customization point object shall satisfy the requirements of `CopyConstructible` (C++Std [copyconstructible]) and `Destructible` (C++Std [destructible]).

All instances of a specific customization point object type shall be equal.

Let t be a (possibly const) customization point object of type T , and $\text{args} \dots$ be a parameter pack expansion of some parameter pack $\text{Args} \dots$. The customization point object t shall be callable as $t(\text{args} \dots)$ when the types of $\text{Args} \dots$ meet the requirements specified in that customization point object's definition. Otherwise, T shall not have a function call operator that participates in overload resolution.

Each customization point object type constrains its return type to satisfy some particular type requirements.

The library defines several named customization point objects. In every translation unit where such a name is defined, it shall refer to the same instance of the customization point object.

[*Note:* Many of the customization points objects in the library evaluate function call expressions with an unqualified name which results in a call to a user-defined function found by argument dependent name lookup (C++Std [basic.lookup.argdep]). To preclude such an expression resulting in a call to unconstrained functions with the same name in namespace `std`, customization point objects specify that lookup for these expressions is performed in a context that includes deleted overloads matching the signatures of overloads defined in namespace `std`. When the deleted overloads are viable, user-defined overloads must be more specialized (C++Std [temp.func.order]) to be used by a customization point object. --end note]

3.1.2 Future requirements

A type F meets the `Future` requirements for some value type T if F is `std::experimental::future<T>` (defined in the C++ Concurrency TS, ISO/IEC TS 19571:2016). [*Note:* This concept is included as a placeholder to be elaborated, with the expectation that the elaborated requirements for `Future` will expand the applicability of some executor customization points. --end note]

3.1.3 ProtoAllocator requirements

A type A meets the `ProtoAllocator` requirements if A is `CopyConstructible` (C++Std [copyconstructible]), `Destructible` (C++Std [destructible]), and `allocator_traits<A>::rebind_alloc<U>` meets the allocator requirements (C++Std [allocator.requirements]), where U is an object type. [*Note:* For example, `std::allocator<void>` meets the proto-allocator requirements but not the allocator requirements. --end note] No comparison operator, copy operation, move operation, or swap operation on these types shall exit via an exception.

3.1.4 ExecutionContext requirements

A type meets the `ExecutionContext` requirements if it satisfies the `EqualityComparable` requirements (C++Std [equalitycomparable]). No comparison operator on these types shall exit via an exception.

3.1.5 BaseExecutor requirements

A type X meets the `BaseExecutor` requirements if it satisfies the requirements of `CopyConstructible` (C++Std [copyconstructible]), `Destructible` (C++Std [destructible]), and `EqualityComparable` (C++Std [equalitycomparable]), as well as the additional requirements listed below.

No comparison operator, copy operation, move operation, swap operation, or member function context on these types shall exit via an exception.

The executor copy constructor, comparison operators, context member function, associated execution functions, and other member functions defined in refinements (TODO: what should this word be?) of the `BaseExecutor` requirements shall not introduce data races as a result of concurrent calls to those functions from different threads.

The destructor shall not block pending completion of the submitted function objects. [Note: The ability to wait for completion of submitted function objects may be provided by the associated execution context. --end note]

In the Table below, `x1` and `x2` denote (possibly `const`) values of type `X`, `mx1` denotes an `xvalue` of type `X`, and `u` denotes an identifier.

(Base executor requirements)		
Expression	Type	Assertion/note/pre-/post-condition
<code>x u(x1);</code>		Shall not exit via an exception. Post: <code>u == x1</code> and <code>u.context() == x1.context()</code> . Shall not exit via an exception.
<code>x u(mx1);</code>		Post: <code>u</code> equals the prior value of <code>mx1</code> and <code>u.context()</code> equals the prior value of <code>mx1.context()</code> . Returns <code>true</code> only if <code>x1</code> and <code>x2</code> can be interchanged with identical effects in any of the expressions defined in these type requirements (TODO and the other executor requirements defined in this Technical Specification). [Note: Returning <code>false</code> does not necessarily imply that the effects are not identical. --end note] operator <code>==</code> shall be reflexive, symmetric, and transitive, and shall not exit via an exception.
<code>x1 == x2</code>	<code>bool</code>	
<code>x1 != x2</code>	<code>bool</code>	Same as <code>!(x1 == x2)</code> .
<code>x1.context()</code>	<code>E& or const E&</code> where <code>E</code> is a type that satisfies the <code>ExecutionContext</code> requirements.	Shall not exit via an exception. The comparison operators and member functions defined in these requirements (TODO and the other executor requirements defined in this Technical Specification) shall not alter the reference returned by this function.

[Commentary: The equality operator is specified primarily as an aid to support postconditions on executor copy construction and move construction. The key word in supporting these postconditions is "interchanged". That is, if a copy is substituted for the original executor it shall produce identical effects, provided the expression, calling context, and program state are otherwise identical. Calls to the copied executor from a different context or program state are not required to produce identical effects, and this is not considered an "interchanged" use of an executor. In particular, even consecutive calls to the same executor need not produce identical effects since the program state has already altered.]

3.1.6 `OneWayExecutor` requirements

The `OneWayExecutor` requirements specify requirements for executors which create execution agents without a channel for awaiting the completion of a submitted function object and obtaining its result. [Note: That is, the executor provides fire-and-forget semantics. --end note]

A type `X` satisfies the `OneWayExecutor` requirements if it satisfies the `BaseExecutor` requirements, as well as the requirements in the table below.

In the Table below, `x` denotes a (possibly `const`) executor object of type `X` and `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))()` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

Expression	Return Type	Operational semantics
		Creates an execution agent which invokes <code>DECAY_COPY(std::forward<F>(f))()</code> at most once, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>execute</code> . May block forward progress of the caller until <code>DECAY_COPY(std::forward<F>(f))()</code> finishes execution.
<code>x.execute(f)</code>	<code>void</code>	The invocation of <code>execute</code> synchronizes with (C++Std [intro.multithread]) the invocation of <code>f</code> . <code>execute</code> shall not propagate any exception thrown by <code>DECAY_COPY(std::forward<F>(f))()</code> or any other function submitted to the executor. [Note: The treatment of exceptions thrown by one-way submitted functions is specific to the concrete executor type. --end note.]

3.1.7 TwoWayExecutor requirements

The TwoWayExecutor requirements specify requirements for executors which creating execution agents with a channel for awaiting the completion of a submitted function object and obtaining its result.

A type X satisfies the TwoWayExecutor requirements if it satisfies the BaseExecutor requirements, as well as the requirements in the table below.

In the Table below, x denotes a (possibly const) executor object of type X , f denotes a function object of type $F\&\&$ callable as `DECAY_COPY(std::forward<F>(f))()` and where `decay_t<F>` satisfies the MoveConstructible requirements, and R denotes the type of the expression `DECAY_COPY(std::forward<F>(f))()`.

Expression	Return Type	Operational semantics
		Creates an execution agent which invokes <code>DECAY_COPY(std::forward<F>(f))()</code> at most once, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>execute</code> .
<code>x.twoway_execute(f)</code>	A type that satisfies the Future requirements for the value type R .	May block forward progress of the caller until <code>DECAY_COPY(std::forward<F>(f))()</code> finishes execution. The invocation of <code>twoway_execute</code> synchronizes with (C++Std [intro.multithread]) the invocation of f. Stores the result of <code>DECAY_COPY(std::forward<F>(f))()</code>, or any exception thrown by <code>DECAY_COPY(std::forward<F>(f))()</code>, in the associated shared state of the resulting Future.

3.1.8 ThenExecutor requirements

The ThenExecutor requirements specify requirements for executors which create execution agents whose initiation is predicated on the readiness of a specified future, and which provide a channel for awaiting the completion of the submitted function object and obtaining its result.

A type X satisfies the ThenExecutor requirements if it satisfies the BaseExecutor requirements, as well as the requirements in the table below.

In the Table below, x denotes a (possibly const) executor object of type X , `pred` denotes a future object satisfying the Future requirements, f denotes a function object of type $F\&\&$ callable as `DECAY_COPY(std::forward<F>(f))(pred)` and where `decay_t<F>` satisfies the MoveConstructible requirements, and R denotes the type of the expression `DECAY_COPY(std::forward<F>(f))(pred)`.

Expression	Return Type	Operational semantics
		When <code>pred</code> is ready, creates an execution agent which invokes <code>DECAY_COPY(std::forward<F>(f))(pred)</code> at most once, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>execute</code> .
<code>x.then_execute(f, pred)</code>	A type that satisfies the Future requirements for the value type R .	May block forward progress of the caller until <code>DECAY_COPY(std::forward<F>(f))(pred)</code> finishes execution. The invocation of <code>then_execute</code> synchronizes with (C++Std [intro.multithread]) the invocation of f. Stores the result of <code>DECAY_COPY(std::forward<F>(f))(pred)</code>, or any exception thrown by <code>DECAY_COPY(std::forward<F>(f))(pred)</code>, in the associated shared state of the resulting Future.

3.1.9 BulkOneWayExecutor requirements

The BulkOneWayExecutor requirements specify requirements for executors which create groups of execution agents in bulk from a single execution function, without a channel for awaiting the completion of the submitted function object invocations and obtaining their result. [Note: That is, the executor provides fire-and-forget semantics. --end note]

A type `X` satisfies the `BulkOneWayExecutor` requirements if it satisfies the `BaseExecutor` requirements, as well as the requirements in the table below.

In the Table below,

- `x` denotes a (possibly `const`) executor object of type `X`,
- `n` denotes a shape object whose type is `executor_shape_t<X>`,
- `sf` denotes a `CopyConstructible` function object with zero arguments whose result type is `S`,
- `i` denotes a (possibly `const`) object whose type is `executor_index_t<X>`,
- `s` denotes an object whose type is `S`,
- `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))(i, s)` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

Expression	Return Type	Operational semantics
<code>x.bulk_execute(f, n, sf)</code>	<code>void</code>	Invokes <code>sf()</code> on an unspecified execution agent to produce the value <code>s</code>. Creates a group of execution agents of shape <code>n</code> which invokes <code>DECAY_COPY(std::forward<F>(f))(i, s)</code> at most once for each value of <code>i</code> in the range <code>[0, n)</code>, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>bulk_execute</code>. May block forward progress of the caller until one or more calls to <code>DECAY_COPY(std::forward<F>(f))(i, s)</code> finish execution. The invocation of <code>bulk_execute</code> synchronizes with <code>(C++Std [intro.multithread])</code> the invocations of <code>f</code>. <code>bulk_execute</code> shall not propagate any exception thrown by <code>DECAY_COPY(std::forward<F>(f))(i, s)</code> or any other function submitted to the executor. [Note: The treatment of exceptions thrown by bulk one-way submitted functions is specific to the concrete executor type. --end note.]

3.1.10 BulkTwoWayExecutor requirements

The `BulkTwoWayExecutor` requirements specify requirements for executors which create groups of execution agents in bulk from a single execution function with a channel for awaiting the completion of a submitted function object invoked by those execution agents and obtaining its result.

A type `X` satisfies the `BulkTwoWayExecutor` requirements if it satisfies the `BaseExecutor` requirements, as well as the requirements in the table below.

In the Table below,

- `x` denotes a (possibly `const`) executor object of type `X`,
- `n` denotes a shape object whose type is `executor_shape_t<X>`,
- `rf` denotes a `CopyConstructible` function object with zero arguments whose result type is `R`,
- `sf` denotes a `CopyConstructible` function object with zero arguments whose result type is `S`,
- `i` denotes a (possibly `const`) object whose type is `executor_index_t<X>`,
- `s` denotes an object whose type is `S`,
- if `R` is non-void,
 - `r` denotes an object whose type is `R`,
 - `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))(i, r, s)` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.
- if `R` is void,
 - `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))(i, s)` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

Expression	Return Type	Operational semantics
<code>x.bulk_twoway_execute(f, n, rf, sf)</code>	A type that satisfies the <code>Future</code> requirements for the value type <code>R</code> .	If <code>R</code> is non-void, invokes <code>rf()</code> on an unspecified execution agent to produce the value <code>r</code>. Invokes <code>sf()</code> on an unspecified execution agent to produce the value <code>s</code>. Creates a group of execution agents of shape <code>n</code> which invokes <code>DECAY_COPY(std::forward<F>(f))(i, r, s)</code> if <code>R</code> is non-void, and otherwise invokes <code>DECAY_COPY(std::forward<F></code>

Expression	Return Type	Operational semantics
		$(f))(i, s)$, at most once for each value of i in the range $[0, n)$, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>bulk_tway_execute</code>. May block forward progress of the caller until one or more invocations of f finish execution. The invocation of <code>bulk_tway_execute</code> synchronizes with (C++Std [intro.multithread]) the invocations of f. Once all invocations of f finish execution, stores r, or any exception thrown by an invocation of f, in the associated shared state of the resulting Future.

3.1.11 BulkThenExecutor requirements

The ThenExecutor requirements specify requirements for executors which create execution agents whose initiation is predicated on the readiness of a specified future, and which provide a channel for awaiting the completion of the submitted function object and obtaining its result.

A type X satisfies the BulkThenExecutor requirements if it satisfies the BaseExecutor requirements, as well as the requirements in the table below.

In the Table below,

- x denotes a (possibly const) executor object of type X ,
- n denotes a shape object whose type is `executor_shape_t<X>`,
- $pred$ denotes a future object satisfying the Future requirements,
- rf denotes a CopyConstructible function object with zero arguments whose result type is R ,
- sf denotes a CopyConstructible function object with zero arguments whose result type is S ,
- i denotes a (possibly const) object whose type is `executor_index_t<X>`,
- s denotes an object whose type is S ,
- if R is non-void,
 - r denotes an object whose type is R ,
 - f denotes a function object of type $F\&\&$ callable as `DECAY_COPY(std::forward<F>(f))(i, pred, r, s)` and where `decay_t<F>` satisfies the MoveConstructible requirements.
- if R is void,
 - f denotes a function object of type $F\&\&$ callable as `DECAY_COPY(std::forward<F>(f))(i, pred, s)` and where `decay_t<F>` satisfies the MoveConstructible requirements.

Expression	Return Type	Operational semantics
<code>x.bulk_then_execute(f, n, pred, rf, sf)</code>	A type that satisfies the Future requirements for the value type R .	If R is non-void, invokes $rf()$ on an unspecified execution agent to produce the value r. Invokes $sf()$ on an unspecified execution agent to produce the value s. When $pred$ is ready, creates a group of execution agents of shape n which invokes <code>DECAY_COPY(std::forward<F>(f))(i, pred, r, s)</code> if R is non-void, and otherwise invokes <code>DECAY_COPY(std::forward<F>(f))(i, pred, s)</code>, at most once for each value of i in the range $[0, n)$, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>bulk_tway_execute</code>. May block forward progress of the caller until one or more invocations of f finish execution. The invocation of <code>bulk_tway_execute</code> synchronizes with (C++Std [intro.multithread]) the invocations of f. Once all invocations of f finish execution, stores r, or any exception thrown by an

Expression	Return Type	Operational semantics
		invocation of <code>f</code> , in the associated shared state of the resulting Future.

3.2 Executor properties

3.2.1 In general

An executor's behavior in generic contexts is determined by a set of executor properties, and each executor property imposes certain requirements on the executor.

An executor's properties are modified by calling the `require` or `prefer` functions. These functions behave according the table below. In the table below, `x` denotes a (possibly const) executor object of type `x`, `*` and `p` denotes a (possibly const) property object.

Expression	Comments
<code>x.require(p)</code> <code>require(x,p)</code>	Returns an executor object with the requested property <code>p</code> added to the set. All other properties of the returned executor are identical to those of <code>x</code> , except where those properties are described below as being mutually exclusive to <code>p</code> . In this case, the mutually exclusive properties are implicitly removed from the set associated with the returned executor.
<code>x.prefer(p)</code> <code>prefer(x,p)</code>	The expression is ill formed if an executor is unable to add the requested property. If the executor is able to add the requested property <code>p</code>, returns an executor object with the requested property added to the set. All other properties of the returned executor are identical to those of <code>x</code>, except where those properties are described below as being mutually exclusive to <code>p</code>. In this case, the mutually exclusive properties are implicitly removed from the set associated with the returned executor. Otherwise, returns a copy of <code>x</code>.

3.2.2 Directionality properties

```
constexpr struct oneway_t {} oneway;
constexpr struct twoway_t {} twoway;
constexpr struct then_t {} then;
```

Property	Requirements
oneway	The executor type satisfies the <code>OneWayExecutor</code> or <code>BulkOneWayExecutor</code> requirements.
twoway	The executor type satisfies the <code>TwoWayExecutor</code> or <code>BulkTwoWayExecutor</code> requirements.
then	The executor type satisfies the <code>ThenExecutor</code> or <code>BulkThenExecutor</code> requirements.

3.2.3 Cardinality properties

```
constexpr struct single_t {} single;
constexpr struct bulk_t {} bulk;
```

Property	Requirements
single	The executor type satisfies the <code>OneWayExecutor</code> , <code>TwoWayExecutor</code> , or <code>ThenExecutor</code> requirements.
bulk	The executor type satisfies the <code>BulkOneWayExecutor</code> , <code>BulkTwoWayExecutor</code> , or <code>BulkThenExecutor</code> requirements.

3.2.4 Blocking properties

```
constexpr struct never_blocking_t {} never_blocking;
constexpr struct possibly_blocking_t {} possibly_blocking;
constexpr struct always_blocking_t {} always_blocking;
```

Property	Requirements
never_blocking	A call to an executor's execution function shall not block pending completion of the execution agents created by that execution function.
possibly_blocking	A call to an executor's execution function may block pending completion of one or more of the execution agents created by that execution function.
always_blocking	A call to an executor's execution function shall block until completion of all execution agents created by that execution function.

The `never_blocking`, `possibly_blocking`, and `always_blocking` properties are mutually exclusive.

3.2.4.1 Properties to indicate if submitted tasks represent continuations

```
constexpr struct continuation_t {} continuation;
constexpr struct not_continuation_t {} not_continuation;
```

Property	Requirements
continuation	Function objects submitted through the executor represent continuations of the caller. If the caller is a lightweight execution

Property	Requirements
	agent managed by the executor or its associated execution context, the execution of the submitted function object may be deferred until the caller completes.
not_continuation	Function objects submitted through the executor do not represent continuations of the caller.

The continuation and not_continuation properties are mutually exclusive.

3.2.5 Properties to indicate likely task submission in the future

```
constexpr struct outstanding_work_t {} outstanding_work;
constexpr struct not_outstanding_work_t {} not_outstanding_work;
```

Property	Requirements
	The existence of the executor object represents an indication of likely future submission of a function object. The executor or its associated execution context may choose to maintain execution resources in anticipation of this submission.
outstanding_work	
	The existence of the executor object does not indicate any likely future submission of a function object.
not_outstanding_work	

The outstanding_work and not_outstanding_work properties are mutually exclusive.

3.2.6 Properties for bulk execution forward progress guarantees

These properties communicate the forward progress and ordering guarantees of execution agents with respect to other agents within the same bulk submission.

```
constexpr struct bulk_sequenced_execution_t {} bulk_sequenced_execution;
constexpr struct bulk_parallel_execution_t {} bulk_parallel_execution;
constexpr struct bulk_unsequenced_execution_t {} bulk_unsequenced_execution;
```

Property	Requirements
bulk_sequenced_execution	
bulk_parallel_execution	
bulk_unsequenced_execution	

TODO: The meanings and relative "strength" of these categories are to be defined. Most of the wording for bulk_sequenced_execution, bulk_parallel_execution, and bulk_unsequenced_execution can be migrated from S 25.2.3 p2, p3, and p4, respectively.

The bulk_sequenced_execution, bulk_parallel_execution, and bulk_unsequenced_execution properties are mutually exclusive.

3.2.7 Properties for mapping of execution on to threads

```
constexpr struct thread_execution_mapping_t {} thread_execution_mapping;
constexpr struct new_thread_execution_mapping_t {} new_thread_execution_mapping;
```

Property	Requirements
thread_execution_mapping	Execution agents created

Property	Requirements
	by the executor are mapped onto threads of execution.
<code>new_thread_execution_mapping</code>	Each execution agent created by the executor is mapped onto a new thread of execution.

The `thread_execution_mapping` and `new_thread_execution_mapping` properties are mutually exclusive.

[*Note:* A mapping of an execution agent onto a thread of execution implies the agent executes as-if on a `std::thread`. Therefore, the facilities provided by `std::thread`, such as thread-local storage, are available. `new_thread_execution_mapping` provides stronger guarantees, in particular that thread-local storage will not be shared between execution agents. --end note]

3.2.8 Properties for customizing memory allocation

```
template<class ProtoAllocator> struct allocator_t { ProtoAllocator alloc; };
template<class ProtoAllocator> constexpr allocator_t<ProtoAllocator> allocator(const ProtoAllocator& a) { return {a}; }
```

Property	Requirements
	Executor implementations shall use the supplied allocator to allocate any memory required to store the submitted function object.
<code>allocator</code>	

3.3 Executor type traits

3.3.1 Determining that a type satisfies executor type requirements

```
template<class T> struct is_executor;
template<class T> struct is_oneway_executor;
template<class T> struct is_twoway_executor;
template<class T> struct is_then_executor;
template<class T> struct is_bulk_oneway_executor;
template<class T> struct is_bulk_twoway_executor;
template<class T> struct is_bulk_then_executor;
```

This sub-clause contains templates that may be used to query the properties of a type at compile time. Each of these templates is a `UnaryTypeTrait` (C++Std [meta.rqmts]) with a `BaseCharacteristic` of `true_type` if the corresponding condition is true, otherwise `false_type`.

Template	Condition	Preconditions
<code>template<class T> struct is_executor</code>	T meets the syntactic requirements for <code>BaseExecutor</code> .	T is a complete type.
<code>template<class T> struct is_oneway_executor</code>	T meets the syntactic requirements for <code>OneWayExecutor</code> .	T is a complete type.
<code>template<class T> struct is_twoway_executor</code>	T meets the syntactic requirements for <code>TwoWayExecutor</code> .	T is a complete type.
<code>template<class T> struct is_then_executor</code>	T meets the syntactic requirements for <code>ThenExecutor</code> .	T is a complete type.
<code>template<class T> struct is_bulk_oneway_executor</code>	T meets the syntactic requirements for <code>BulkOneWayExecutor</code> .	T is a complete type.
<code>template<class T> struct is_bulk_twoway_executor</code>	T meets the syntactic requirements for <code>BulkTwoWayExecutor</code> .	T is a complete type.
<code>template<class T> struct is_bulk_then_executor</code>	T meets the syntactic requirements for <code>BulkThenExecutor</code> .	T is a complete type.

3.3.2 Associated execution context type

```
template<class Executor>
struct executor_context
{
    using type = std::decay_t<decltype(declval<const Executor&>().context())>;
};
```

3.3.3 Associated future type

```
template<class Executor, class T>
struct executor_future
{
    using type = decltype(execution::require(declval<const Executor&>(), execution::twoway).twoway_execute(declval<T(*)>()));
};
```

3.3.4 Associated shape type

```
template<class Executor>
struct executor_shape
{
private:
    // exposition only
    template<class T>
    using helper = typename T::shape_type;

public:
    using type = std::experimental::detected_or_t<
        size_t, helper, decltype(execution::require(declval<const Executor&>(), execution::bulk))
    >;

    // exposition only
    static_assert(std::is_integral_v<type>, "shape type must be an integral type");
};
```

3.3.5 Associated index type

```
template<class Executor>
struct executor_index
{
private:
    // exposition only
    template<class T>
    using helper = typename T::index_type;

public:
    using type = std::experimental::detected_or_t<
        executor_shape_t<Executor>, helper, decltype(execution::require(declval<const Executor&>(), execution::bulk))
    >;

    // exposition only
    static_assert(std::is_integral_v<type>, "index type must be an integral type");
};
```

3.3.6 Member detection type traits for properties

```
template<class Executor, class Property> struct has_require_member;
template<class Executor, class Property> struct has_prefer_member;
```

This sub-clause contains templates that may be used to query the properties of a type at compile time. Each of these templates is a UnaryTypeTrait (C++Std [meta.rqmts]) with a BaseCharacteristic of true_type if the corresponding condition is true, otherwise false_type.

Template	Condition	Preconditions
template<class T> struct has_require_member	The expression declval<const Executor>().require(declval<Property>()) is well formed.	T is a complete type.
template<class T> struct has_prefer_member	The expression declval<const Executor>().prefer(declval<Property>()) is well formed.	T is a complete type.

3.3.7 Member return type traits for properties

```
template<class Executor, class Property> struct require_member_result;
template<class Executor, class Property> struct prefer_member_result;
```

This sub-clause contains templates that may be used to query the properties of a type at compile time. Each of these templates is a TransformationTrait (C++Std [meta.rqmts]).

Template	Condition	Comments
template<class T> struct require_member_result	The expression declval<const Executor>().require(declval<Property>()) is well formed.	The member typedef type shall name the type of the expression declval<const Executor>().require(declval<Property>()).
template<class T> struct prefer_member_result	The expression declval<const Executor>().prefer(declval<Property>()) is	The member typedef type shall name the type of the

Template	Condition	Comments
	well formed.	expression declval<const Executor>().prefer(declval<Property>()).

3.4 Executor customization points

Executor customization points are functions which adapt an executor's properties. Executor customization points enable uniform use of executors in generic contexts.

When an executor customization point named *NAME* invokes a free execution function of the same name, overload resolution is performed in a context that includes the declaration `void NAME(auto&... args) = delete;`, where `sizeof...(args)` is the arity of the free execution function. This context also does not include a declaration of the executor customization point.

[*Note:* This provision allows executor customization points to call the executor's free, non-member execution function of the same name without recursion. --end note]

Whenever `std::experimental::concurrency_v2::execution::NAME(ARGS)` is a valid expression, that expression satisfies the syntactic requirements for the free execution function named *NAME* with arity `sizeof...(ARGS)` with that free execution function's semantics.

3.4.1 require

```
namespace {
    constexpr unspecified prefer = unspecified;
}
```

The name *require* denotes a customization point. The effect of the expression `std::experimental::concurrency_v2::execution::require(E, P0, Pn...)` for some expressions *E* and *P0*, and where *Pn...* represents *N* expressions (where *N* is 0 or more), is equivalent to:

- `(E).require(P0)` if *N* == 0 and `has_require_member_v<decay_t<decltype(E)>, decltype(P0)>` is true.
- Otherwise, `require(E, P0)` if *N* == 0 and the expression is well formed.
- Otherwise, *E* if *N* == 0 and:
 - `is_same<decay_t<decltype(P0)>, oneway_t>` is true and `(is_oneway_executor_v<decay_t<decltype(E)>> || is_bulk_oneway_executor_v<decay_t<decltype(E)>>)` is true; or
 - `is_same<decay_t<decltype(P0)>, twoway_t>` is true and `(is_twoway_executor_v<decay_t<decltype(E)>> || is_bulk_twoway_executor_v<decay_t<decltype(E)>>)` is true; or
 - `is_same<decay_t<decltype(P0)>, single_t>` is true and `(is_oneway_executor_v<decay_t<decltype(E)>> || is_twoway_executor_v<decay_t<decltype(E)>>)` is true; or
 - `is_same<decay_t<decltype(P0)>, bulk_t>` is true and `(is_bulk_oneway_executor_v<decay_t<decltype(E)>> || is_bulk_twoway_executor_v<decay_t<decltype(E)>>)` is true; or
- `is_same<decay_t<decltype(P0)>, possibly_blocking_t>` is true.
- Otherwise, a value *E1* of unspecified type that holds a copy of *E* if *N* == 0 and `is_same<decay_t<decltype(P0)>, twoway_t>` is true. The type of *E1* satisfies the *BaseExecutor* requirements and provides member functions such that calls to `require`, `prefer`, `context`, `execute` and `bulk_execute` are forwarded to the corresponding member functions of the copy of *E*, if present. In addition, if *E* satisfies the *OnewayExecutor* requirements, *E1* shall satisfy the *TwowayExecutor* requirements by implementing `twoway_execute` in terms of `execute`. Similarly, if *E* satisfies the *BulkOnewayExecutor* requirements, *E1* shall satisfy the *BulkTwowayExecutor* requirements by implementing `bulk_twoway_execute` in terms of `bulk_execute`. For some type *T*, the type yielded by `ExecutorFuture_t<decltype(E1), T>` is `std::experimental::future<T>`. *E1* has the same executor properties as *E*, except for the addition of the `twoway_t` property.
- Otherwise, a value *E1* of unspecified type that holds a copy of *E* if *N* == 0 and `is_same<decay_t<decltype(P0)>, bulk_t>` is true. The type of *E1* satisfies the *BaseExecutor* requirements and provides member functions such that calls to `require`, `prefer`, `context`, `execute` and `twoway_execute` are forwarded to the corresponding member functions of the copy of *E*, if present. In addition, if *E* satisfies the *OnewayExecutor* requirements, *E1* shall satisfy the *BulkExecutor* requirements by implementing `bulk_execute` in terms of `execute`. If *E* also satisfies the *TwowayExecutor* requirements, *E1* shall satisfy the *BulkTwowayExecutor* requirements by implementing `bulk_twoway_execute` in terms of `bulk_execute`. *E1* has the same executor properties as *E*, except for the addition of the `bulk_t` property.
- Otherwise, a value *E1* of unspecified type that holds a copy of *E* if *N* == 0 and `is_same<decay_t<decltype(P0)>, always_blocking_t>` is true. The type of *E1* satisfies the *BaseExecutor* requirements and provides member functions such that calls to `require`, `prefer`, `context`, `execute`, `twoway_execute`, `bulk_execute`, and `bulk_twoway_execute` are forwarded to the corresponding member functions of the copy of *E*, if present. In addition, *E1* provides an overload of `require` that returns a copy of *E1*, and all functions `execute`, `twoway_execute`, `bulk_execute`, and `bulk_twoway_execute` shall block the calling thread until the submitted functions have finished execution. *E1* has the same executor properties as *E*, except for the addition of the `always_blocking_t` property, and removal of `never_blocking_t` and `possibly_blocking_t` properties if present.
- Otherwise, `std::experimental::concurrency_v2::execution::require(std::experimental::concurrency_v2::execution::require(E, P0), Pn...)` if *N* > 0 and the expression is well formed.
- Otherwise, `std::experimental::concurrency_v2::execution::require(E, P0, Pn...)` is ill-formed.

3.4.2 prefer

```
namespace {
    constexpr unspecified prefer = unspecified;
}
```

The name `prefer` denotes a customization point. The effect of the expression `std::experimental::concurrency_v2::execution::prefer(E, P0, Pn...)` for some expressions `E` and `P0`, and where `Pn...` represents `N` expressions (where `N` is 0 or more), is equivalent to:

- `(E).prefer(P0)` if `N == 0` and `has_prefer_member_v<decay_t<decltype(E)>, decltype(P0)>` is true.
- Otherwise, `(E).require(P0)` if `N == 0` and `has_require_member_v<decay_t<decltype(E)>, decltype(P0)>` is true.
- Otherwise, `prefer(E, P0)` if `N == 0` and the expression is well formed.
- Otherwise, `E` if `N == 0`.
- Otherwise, `std::experimental::concurrency_v2::execution::prefer( std::experimental::concurrency_v2::execution::prefer(E, P0), Pn...)` if the expression is well formed.
- Otherwise, `std::experimental::concurrency_v2::execution::prefer(E, P0, Pn...)` is ill-formed.

3.4.3 Customization point type traits

```
template<class Executor, class... Properties> struct can_require;
template<class Executor, class... Properties> struct can_prefer;
```

This sub-clause contains templates that may be used to query the properties of a type at compile time. Each of these templates is a `UnaryTypeTrait` (C++Std [meta.rqmts]) with a `BaseCharacteristic` of `true_type` if the corresponding condition is true, otherwise `false_type`.

Template	Condition	Preconditions
<pre>template<class T> struct can_require</pre>	The expression <code>std::experimental::concurrency_v2::execution::require(declval<const Executor>(), declval<Properties>()...)</code> is well formed.	T is a complete type.
<pre>template<class T> struct can_prefer</pre>	The expression <code>std::experimental::concurrency_v2::execution::prefer(declval<const Executor>(), declval<Properties>()...)</code> is well formed.	T is a complete type.

3.5 Polymorphic executor wrappers

3.5.1 Class `bad_executor`

An exception of type `bad_executor` is thrown by executor member functions `execute`, `twoway_execute`, `bulk_execute`, and `bulk_twoway_execute` when the executor object has no target.

```
class bad_executor : public exception
{
public:
    // constructor:
    bad_executor() noexcept;
};
```

3.5.1.1 `bad_executor` constructors

```
bad_executor() noexcept;
```

Effects: Constructs a `bad_executor` object.

Postconditions: `what()` returns an implementation-defined NTBS.

3.5.2 Class `executor`

The `executor` class provides a polymorphic wrapper for executor types.

```
class executor
{
public:
    class context_type;

    // construct / copy / destroy:

    executor() noexcept;
```



```

executor(nullptr_t) noexcept;
executor(const executor& e) noexcept;
executor(executor&& e) noexcept;
template<class Executor> executor(Executor e);

executor& operator=(const executor& e) noexcept;
executor& operator=(executor&& e) noexcept;
executor& operator=(nullptr_t) noexcept;
template<class Executor> executor& operator=(Executor e);

~executor();

// executor modifiers:

void swap(executor& other) noexcept;

// executor operations:

const context_type& context() const noexcept;

executor require(oneway_t) const;
executor require(twoway_t) const;
executor require(single_t) const;
executor require(bulk_t) const;
executor require(thread_execution_mapping_t) const;
executor require(never_blocking_t p) const;
executor require(possibly_blocking_t p) const;
executor require(always_blocking_t p) const;

executor prefer(continuation_t p) const;
executor prefer(not_continuation_t p) const;
executor prefer(outstanding_work_t p) const;
executor prefer(not_outstanding_work_t p) const;
executor prefer(bulk_sequenced_execution_t p) const;
executor prefer(bulk_parallel_execution_t p) const;
executor prefer(bulk_unsequenced_execution_t p) const;
executor prefer(new_thread_execution_mapping_t p) const;
template <class Property> executor prefer(const Property& p) const;

template<class Function>
    void execute(Function&& f) const;

template<class Function>
    std::experimental::future<result_of_t<decay_t<Function>()>>
        twoway_execute(Function&& f) const

template<class Function, class SharedFactory>
    void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;

template<class Function, class ResultFactory, class SharedFactory>
    std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
        void bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf) const;

// executor capacity:

explicit operator bool() const noexcept;

// executor target access:

const type_info& target_type() const noexcept;
template<class Executor> Executor* target() noexcept;
template<class Executor> const Executor* target() const noexcept;
};

// executor comparisons:

bool operator==(const executor& a, const executor& b) noexcept;
bool operator==(const executor& e, nullptr_t) noexcept;
bool operator==(nullptr_t, const executor& e) noexcept;
bool operator!=(const executor& a, const executor& b) noexcept;
bool operator!=(const executor& e, nullptr_t) noexcept;
bool operator!=(nullptr_t, const executor& e) noexcept;

// executor specialized algorithms:

void swap(executor& a, executor& b) noexcept;

```

The executor class satisfies the BaseExecutor, DefaultConstructible (C++Std [defaultconstructible]), and CopyAssignable (C++Std [copyassignable]) requirements.

[Note: To meet the noexcept requirements for executor copy constructors and move constructors, implementations may share a target between two or more executor objects. --end note]

The *target* is the executor object that is held by the wrapper.

3.5.2.1 executor constructors

executor() noexcept;

Postconditions: !*this.

executor(nullptr_t) noexcept;

Postconditions: !*this.

executor(const executor& e) noexcept;

Postconditions: !*this if !e; otherwise, *this targets e.target() or a copy of e.target().

executor(executor&& e) noexcept;

Effects: If !e, *this has no target; otherwise, moves e.target() or move-constructs the target of e into the target of *this, leaving e in a valid state with an unspecified value.

template<class Executor> executor(Executor e);

Requires:

- can_require_v<Executor, oneway>
- can_require_v<Executor, twoway>
- can_require_v<Executor, single>
- can_require_v<Executor, bulk>
- can_require_v<Executor, never_blocking>
- can_require_v<Executor, possibly_blocking>
- can_require_v<Executor, always_blocking>
- can_prefer_v<Executor, continuation>
- can_prefer_v<Executor, not_continuation>
- can_prefer_v<Executor, outstanding_work>
- can_prefer_v<Executor, not_outstanding_work>
- can_prefer_v<Executor, bulk_sequenced_execution>
- can_prefer_v<Executor, bulk_parallel_execution>
- can_prefer_v<Executor, bulk_unsequenced_execution>
- can_require_v<Executor, thread_execution_mapping>
- can_prefer_v<Executor, new_thread_execution_mapping>

Effects: *this targets a copy of e initialized with std::move(e).

3.5.2.2 executor assignment

executor& operator=(const executor& e) noexcept;

Effects: executor(e).swap(*this).

Returns: *this.

executor& operator=(executor&& e) noexcept;

Effects: Replaces the target of *this with the target of e, leaving e in a valid state with an unspecified value.

Returns: *this.

executor& operator=(nullptr_t) noexcept;

Effects: executor(nullptr).swap(*this).

Returns: *this.

template<class Executor> executor& operator=(Executor e);

Requires: As for template<class Executor> executor(Executor e).

Effects: executor(std::move(e)).swap(*this).

Returns: *this.

3.5.2.3 executor destructor

~executor();

Effects: If *this != nullptr, releases shared ownership of, or destroys, the target of *this.

3.5.2.4 executor modifiers

```
void swap(executor& other) noexcept;
```

Effects: Interchanges the targets of *this and other.

3.5.2.5 executor operations

```
const context_type& context() const noexcept;
```

Requires: *this != nullptr.

Returns: A polymorphic execution context wrapper whose target is e.context(), where e is the target object of *this.

```
executor require(oneway_t) const;
executor require(twoway_t) const;
executor require(single_t) const;
executor require(bulk_t) const;
executor require(thread_execution_mapping_t) const;
```

Returns: *this.

```
executor require(never_blocking_t p) const;
executor require(possibly_blocking_t p) const;
executor require(always_blocking_t p) const;
```

Returns: A polymorphic wrapper whose target is execution::require(e, p), where e is the target object of *this.

```
executor prefer(continuation_t) const;
executor prefer(not_continuation_t) const;
executor prefer(outstanding_work_t) const;
executor prefer(not_outstanding_work_t) const;
executor prefer(bulk_sequenced_execution_t) const;
executor prefer(bulk_parallel_execution_t) const;
executor prefer(bulk_unsequenced_execution_t) const;
executor prefer(new_thread_execution_mapping_t) const;
```

Returns: A polymorphic wrapper whose target is execution::prefer(e, p), where e is the target object of *this.

```
template <class Property> executor prefer(const Property& p) const;
```

Returns: this->require(p) if that expression is well formed, otherwise *this.

```
template<class Function>
void execute(Function&& f) const;
```

Effects: Performs e2.execute(f2), where:

- e1 is the target object of *this;
- e2 is the result of require(require(e1, single), oneway);
- f1 is the result of DECAY_COPY(std::forward<Function>(f));
- f2 is a function object of unspecified type that, when called as f2(), performs f1().

```
template<class Function>
std::experimental::future<result_of_t<decay_t<Function>()>>
twoway_execute(Function&& f) const
```

Effects: Performs e2.twoway_execute(f2), where:

- e1 is the target object of *this;
- e2 is the result of require(require(e1, single), twoway);
- f1 is the result of DECAY_COPY(std::forward<Function>(f));
- f2 is a function object of unspecified type that, when called as f2(), performs f1().

Returns: A future, whose shared state is made ready when the future returned by e.twoway_execute(f2) is made ready, containing the result of f1() or any exception thrown by f1(). [Note: e2.twoway_execute(f2) may return any future type that satisfies the Future requirements, and not necessarily std::experimental::future. One possible implementation approach is for the polymorphic wrapper to attach a continuation to the inner future via that object's then() member function. When invoked, this continuation stores the result in the outer future's associated shared and makes that shared state ready. --end note]

```
template<class Function, class SharedFactory>
void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;
```

Effects: Performs e2.bulk_execute(f2, n, sf2), where:

- e1 is the target object of *this;
- e2 is the result of require(require(e1, bulk), oneway);
- sf1 is the result of DECAY_COPY(std::forward<SharedFactory>(sf));

- sf2 is a function object of unspecified type that, when called as sf2(), performs sf1();
- s1 is the result of sf1();
- s2 is the result of sf2();
- f1 is the result of DECAY_COPY(std::forward<Function>(f));
- f2 is a function object of unspecified type that, when called as f2(i, s2), performs f1(i, s1), where i is a value of type size_t.

```
template<class Function, class ResultFactory, class SharedFactory>
    std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
        void bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf) const;
```

Effects: Performs e.bulk_twoway_execute(f2, n, rf2, sf2), where:

- e1 is the target object of *this;
- e2 is the result of require(require(e1, bulk), twoway);
- rf1 is the result of DECAY_COPY(std::forward<ResultFactory>(rf));
- rf2 is a function object of unspecified type that, when called as rf2(), performs rf1();
- sf1 is the result of DECAY_COPY(std::forward<SharedFactory>(sf));
- sf2 is a function object of unspecified type that, when called as sf2(), performs sf1();
- if decltype(rf1()) is non-void, r1 is the result of rf1();
- if decltype(rf2()) is non-void, r2 is the result of rf2();
- s1 is the result of sf1();
- s2 is the result of sf2();
- f1 is the result of DECAY_COPY(std::forward<Function>(f));
- if decltype(rf1()) is non-void and decltype(rf2()) is non-void, f2 is a function object of unspecified type that, when called as f2(i, r2, s2), performs f1(i, r1, s1), where i is a value of type size_t.
- if decltype(rf1()) is non-void and decltype(rf2()) is void, f2 is a function object of unspecified type that, when called as f2(i, s2), performs f1(i, r1, s1), where i is a value of type size_t.
- if decltype(rf1()) is void and decltype(rf2()) is non-void, f2 is a function object of unspecified type that, when called as f2(i, r2, s2), performs f1(i, s1), where i is a value of type size_t.
- if decltype(rf1()) is void and decltype(rf2()) is void, f2 is a function object of unspecified type that, when called as f2(i, s2), performs f1(i, s1), where i is a value of type size_t.

Returns: A future, whose shared state is made ready when the future returned by e.bulk_twoway_execute(f2, n, rf2, sf2) is made ready, containing the result in r1 (if decltype(rf1()) is non-void) or any exception thrown by an invocation f1. [Note: e2.bulk_twoway_execute(f2) may return any future type that satisfies the Future requirements, and not necessarily std::experimental::future. One possible implementation approach is for the polymorphic wrapper to attach a continuation to the inner future via that object's then() member function. When invoked, this continuation stores the result in the outer future's associated shared and makes that shared state ready. --end note]

3.5.2.6 executor capacity

```
explicit operator bool() const noexcept;
```

Returns: true if *this has a target, otherwise false.

3.5.2.7 executor target access

```
const type_info& target_type() const noexcept;
```

Returns: If *this has a target of type T, typeid(T); otherwise, typeid(void).

```
template<class Executor> Executor* target() noexcept;
template<class Executor> const Executor* target() const noexcept;
```

Returns: If target_type() == typeid(Executor) a pointer to the stored executor target; otherwise a null pointer value.

3.5.2.8 executor comparisons

```
bool operator==(const executor& a, const executor& b) noexcept;
```

Returns:

- true if !a and !b;
- true if a and b share a target;
- true if e and f are the same type and e == f, where e is the target of a and f is the target of b;
- otherwise false.

```
bool operator==(const executor& e, nullptr_t) noexcept;
bool operator==(nullptr_t, const executor& e) noexcept;
```

Returns: !e.

```
bool operator!=(const executor& a, const executor& b) noexcept;
```

Returns: !(a == b).

```
bool operator!=(const executor& e, nullptr_t) noexcept;
bool operator!=(nullptr_t, const executor& e) noexcept;
```

Returns: (bool) e.

3.5.2.9 executor specialized algorithms

```
void swap(executor& a, executor& b) noexcept;
```

Effects: a.swap(b).

3.5.3 Class executor::context_type

The executor::context_type class provides a polymorphic wrapper for the execution context associated with a polymorphic executor.

```
class executor::context_type
{
public:
    context_type(const context_type&) = delete;
    context_type& operator=(const context_type&) = delete;
};

// executor context_type comparisons:

bool operator==(const executor::context_type& a, const executor::context_type& b) noexcept;
template<class Context> operator==(const executor::context_type& a, const Context& b) noexcept;
template<class Context> operator==(const Context& a, const executor::context_type& b) noexcept;
bool operator!=(const executor::context_type& a, const executor::context_type& b) noexcept;
template<class Context> operator!=(const executor::context_type& a, const Context& b) noexcept;
template<class Context> operator!=(const Context& a, const executor::context_type& b) noexcept;
```

The *target* is the execution context that is associated with the target of the executor that created the context_type wrapper.

3.5.3.1 executor::context_type comparisons

```
bool operator==(const executor::context_type& a, const executor::context_type& b) noexcept;
```

Returns: - true if !a and !b; - true if a and b share a target; - true if c and d are the same type and c == d, where c is the target of a and d is the target of b; - otherwise false.

```
template<class Context> operator==(const executor::context_type& a, const Context& b) noexcept;
```

Returns: - true if c and b are the same type and c == b, where c is the target of a; - otherwise false.

```
template<class Context> operator==(const Context& a, const executor::context_type& b) noexcept;
```

Returns: b == a.

```
bool operator!=(const executor::context_type& a, const executor::context_type& b) noexcept;
template<class Context> operator!=(const executor::context_type& a, const Context& b) noexcept;
template<class Context> operator!=(const Context& a, const executor::context_type& b) noexcept;
```

Returns: !(a == b).

3.6 Thread pools

Thread pools create execution agents which execute on threads without incurring the overhead of thread creation and destruction whenever such agents are needed.

3.6.1 Header <thread_pool> synopsis

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class static_thread_pool;

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

3.6.2 Class static_thread_pool

static_thread_pool is a statically-sized thread pool which may be explicitly grown via thread attachment. The static_thread_pool is expected to be created with the use case clearly in mind with the number of threads known by the creator. As a result, no default constructor is considered correct for arbitrary use cases and static_thread_pool does not support any form of automatic resizing.

`static_thread_pool` presents an effectively unbounded input queue and the execution functions of `static_thread_pool`'s associated executors do not block on this input queue.

[*Note:* Because `static_thread_pool` provides parallel execution agents, situations which require concurrent execution properties are not guaranteed correctness. --end note.]

```
class static_thread_pool
{
public:
    using executor_type = see-below;

    // construction/destruction
    explicit static_thread_pool(std::size_t num_threads);

    // nocopy
    static_thread_pool(const static_thread_pool&) = delete;
    static_thread_pool& operator=(const static_thread_pool&) = delete;

    // stop accepting incoming work and wait for work to drain
    ~static_thread_pool();

    // attach current thread to the thread pools list of worker threads
    void attach();

    // signal all work to complete
    void stop();

    // wait for all threads in the thread pool to complete
    void wait();

    // placeholder for a general approach to getting executors from
    // standard contexts.
    executor_type executor() noexcept;
};

bool operator==(const static_thread_pool& a, const static_thread_pool& b) noexcept;
bool operator!=(const static_thread_pool& a, const static_thread_pool& b) noexcept;
```

The class `static_thread_pool` satisfies the `ExecutionContext` requirements.

For an object of type `static_thread_pool`, *outstanding work* is defined as the sum of:

- the number of existing executor objects associated with the `static_thread_pool` for which the `execution::outstanding_work` property is established;
- the number of function objects that have been added to the `static_thread_pool` via the `static_thread_pool` executor, but not yet executed; and
- the number of function objects that are currently being executed by the `static_thread_pool`.

The `static_thread_pool` member functions `executor`, `attach`, `wait`, and `stop`, and the associated executors' copy constructors and member functions, do not introduce data races as a result of concurrent calls to those functions from different threads of execution.

3.6.2.1 Types

```
using executor_type = see-below;
```

An executor type conforming to the specification for `static_thread_pool` executor types described below.

3.6.2.2 Construction and destruction

```
static_thread_pool(std::size_t num_threads);
```

Effects: Constructs a `static_thread_pool` object with `num_threads` threads of execution, as if by creating objects of type `std::thread`.

```
~static_thread_pool();
```

Effects: Destroys an object of class `static_thread_pool`. Performs `stop()` followed by `wait()`.

3.6.2.3 Worker management

```
void attach();
```

Effects: Adds the calling thread to the pool such that this thread is used to execute submitted function objects. [*Note:* Threads created during thread pool construction, or previously attached to the pool, will continue to be used for function object execution. --end note] Blocks the calling thread until signalled to complete by `stop()` or `wait()`, and then blocks until all the threads created during `static_thread_pool` object construction have completed. (NAMING: a possible alternate name for this function is `join()`.)

```
void stop();
```

Effects: Signals the threads in the pool to complete as soon as possible. If a thread is currently executing a function object, the thread will exit only after completion of that function object. The call to `stop()` returns without waiting for the threads to complete. Subsequent calls to `attach` complete immediately.

```
void wait();
```

Effects: If not already stopped, signals the threads in the pool to complete once the outstanding work is 0. Blocks the calling thread (C++Std [defs.block]) until all threads in the pool have completed, without executing submitted function objects in the calling thread. Subsequent calls to `attach` complete immediately.

Synchronization: The completion of each thread in the pool synchronizes with (C++Std [intro.multithread]) the corresponding successful `wait()` return.

3.6.2.4 Executor creation

```
executor_type executor() noexcept;
```

Returns: An executor that may be used to submit function objects to the thread pool. The returned executor has the following properties already established:

- `execution::oneway`
- `execution::twoway`
- `execution::then`
- `execution::single`
- `execution::bulk`
- `execution::possibly_blocking`
- `execution::not_continuation`
- `execution::not_outstanding_work`
- `execution::allocator(std::allocator<void>())`

3.6.2.5 Comparisons

```
bool operator==(const static_thread_pool& a, const static_thread_pool& b) noexcept;
```

Returns: `std::addressof(a) == std::addressof(b)`.

```
bool operator!=(const static_thread_pool& a, const static_thread_pool& b) noexcept;
```

Returns: `!(a == b)`.

3.6.3 static_thread_pool executor types

All executor types accessible through `static_thread_pool::executor()`, and subsequent calls to the member functions require and prefer, conform to the following specification.

```
class C
{
public:
    // types:

    typedef std::size_t shape_type;
    typedef std::size_t index_type;

    // construct / copy / destroy:

    C(const C& other) noexcept;
    C(C&& other) noexcept;

    C& operator=(const C& other) noexcept;
    C& operator=(C&& other) noexcept;

    // executor operations:

    C require(execution::oneway_t) const;
    C require(execution::twoway_t) const;
    C require(execution::then_t) const;
    C require(execution::single_t) const;
    C require(execution::bulk_t) const;
    C require(execution::bulk_parallel_execution_t) const;
    C require(execution::thread_execution_mapping_t) const;
    see-below require(execution::never_blocking_t) const;
    see-below require(execution::possibly_blocking_t) const;
    see-below require(execution::always_blocking_t) const;
    see-below require(execution::continuation_t) const;
    see-below require(execution::not_continuation_t) const;
    see-below require(execution::outstanding_work_t) const;
```

```

see-below require(execution::not_outstanding_work_t) const;
template<class ProtoAllocator>
    see-below require(const execution::allocator_t<ProtoAllocator>& a) const;

template<class Property> see-below prefer(const Property& p) const;

bool running_in_this_thread() const noexcept;

static_thread_pool& context() const noexcept;

template<class Function>
    void execute(Function&& f) const;

template<class Function>
    std::experimental::future<result_of_t<decay_t<Function>()>>
        twoway_execute(Function&& f) const

template<class Function, class Future>
    std::experimental::future<result_of_t<decay_t<Function>(decay_t<Future>)>>
        then_execute(Function&& f, Future&& pred) const;

template<class Function, class SharedFactory>
    void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;

template<class Function, class ResultFactory, class SharedFactory>
    std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
        void bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf) const;
};

bool operator==(const C& a, const C& b) noexcept;
bool operator!=(const C& a, const C& b) noexcept;

```

C is a type satisfying the BaseExecutor, OneWayExecutor, TwoWayExecutor, BulkOneWayExecutor, and BulkTwoWayExecutor requirements. Objects of type C are associated with a static_thread_pool.

3.6.3.1 Constructors

C(const C& other) noexcept;

Postconditions: *this == other.

C(C&& other) noexcept;

Postconditions: *this is equal to the prior value of other.

3.6.3.2 Assignment

C& operator=(const C& other) noexcept;

Postconditions: *this == other.

Returns: *this.

C& operator=(C&& other) noexcept;

Postconditions: *this is equal to the prior value of other.

Returns: *this.

3.6.3.3 Operations

```

C require(execution::oneway_t) const;
C require(execution::twoway_t) const;
C require(execution::then_t) const;
C require(execution::single_t) const;
C require(execution::bulk_t) const;
C require(execution::bulk_parallel_execution_t) const;
C require(execution::thread_execution_mapping_t) const;

```

Returns: *this.

```

see-below require(execution::never_blocking_t) const;
see-below require(execution::possibly_blocking_t) const;
see-below require(execution::always_blocking_t) const;
see-below require(execution::continuation_t) const;
see-below require(execution::not_continuation_t) const;
see-below require(execution::outstanding_work_t) const;
see-below require(execution::not_outstanding_work_t) const;

```


Returns: An executor object of an unspecified type conforming to these specifications, associated with the same thread pool as **this*, and having the requested property established. When the requested property is part of a group that is defined as a mutually exclusive set, any other properties in the group are removed from the returned executor object. All other properties of the returned executor object are identical to those of **this*.

```
template<class ProtoAllocator>
    see-below require(const execution::allocator_t<ProtoAllocator>& a) const;
```

Returns: An executor object of an unspecified type conforming to these specifications, associated with the same thread pool as **this*, with the `execution::allocator_t<ProtoAllocator>` property established such that allocation and deallocation associated with function submission will be performed using a copy of `a.alloc`. All other properties of the returned executor object are identical to those of **this*.

```
template<class Property> see-below prefer(const Property& p) const;
```

Returns: `this->require(p)` if that expression is well formed, otherwise **this*.

```
bool running_in_this_thread() const noexcept;
```

Returns: true if the current thread of execution is a thread that was created by or attached to the associated `static_thread_pool` object.

```
static_thread_pool& context() const noexcept;
```

Returns: A reference to the associated `static_thread_pool` object.

```
template<class Function>
    void execute(Function&& f) const;
```

Effects: Submits the function `f` for execution on the `static_thread_pool` according to the `OneWayExecutor` requirements and the properties established for **this*. If the submitted function `f` exits via an exception, the `static_thread_pool` calls `std::terminate()`.

```
template<class Function>
    std::experimental::future<result_of_t<decay_t<Function>()>>
        twoway_execute(Function&& f) const
```

Effects: Submits the function `f` for execution on the `static_thread_pool` according to the `TwoWayExecutor` requirements and the properties established for **this*.

Returns: A future with behavior as specified by the `TwoWayExecutor` requirements.

```
template<class Function, class Future>
    std::experimental::future<result_of_t<decay_t<Function>(decay_t<Future>)>>
        then_execute(Function&& f, Future&& pred) const
```

Effects: Submits the function `f` for execution on the `static_thread_pool` according to the `ThenExecutor` requirements and the properties established for **this*.

Returns: A future with behavior as specified by the `ThenExecutor` requirements.

```
template<class Function, class SharedFactory>
    void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;
```

Effects: Submits the function `f` for bulk execution on the `static_thread_pool` according to the `BulkOneWayExecutor` requirements and the properties established for **this*. If the submitted function `f` exits via an exception, the `static_thread_pool` calls `std::terminate()`.

```
template<class Function, class ResultFactory, class SharedFactory>
    std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
        void bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf) const;
```

Effects: Submits the function `f` for bulk execution on the `static_thread_pool` according to the `BulkTwoWayExecutor` requirements and the properties established for **this*.

Returns: A future with behavior as specified by the `BulkTwoWayExecutor` requirements.

```
template<class Function, class Future, class ResultFactory, class SharedFactory>
    std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
        void bulk_then_execute(Function&& f, Future&& pred, size_t n, ResultFactory&& rf, SharedFactory&& sf) const;
```

Effects: Submits the function `f` for bulk execution on the `static_thread_pool` according to the `BulkThenExecutor` requirements and the properties established for **this*.

Returns: A future with behavior as specified by the `BulkThenExecutor` requirements.

3.6.3.4 Comparisons

```
bool operator==(const C& a, const C& b) noexcept;
```

Returns: true if `a.context() == b.context()` and `a` and `b` have identical properties, otherwise false.

```
bool operator!=(const C& a, const C& b) noexcept;
```

Returns: !(a == b).